| STUDENT NAME: AMANDEEP | MODULE: My Practice | CODE: JAVA_Q017_COMP |
|---|---|---|
| BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02 | COURSE: OOPS Concepts using Java with AI | STATUS: In Progress |

# Practice Worksheet – Q017 – Vehicle Computation & Fleet Details

Formulate a commercial fleet computation engine calculating fuel efficiency, payload load factor, and total operating cost per kilometer.

**KEY LEARNING OBJECTIVES:**

- Model vehicle telemetry and operating expense calculations.
- Apply SOLID object-oriented design principles and access control.
- Implement defensive parameter validation and error-free edge handling.
- Verify correctness with standard JUnit test assertions and execution logs.

**STEP-BY-STEP WORKSHEET INSTRUCTIONS:**

**Step 1:** Review the business domain requirements and model the class structure.
**Step 2:** Implement private attributes, constructor overloading, and accessors.
**Step 3:** Implement the core computation and validation methods in clean Java.
**Step 4:** Document test execution examples and verify edge-case behavior.

## EXERCISE 17.1: DOMAIN MODELING & CLASS ARCHITECTURE (FLEET VEHICLE CLASS MODEL)

**Problem Statement:** Define the core class architecture for Q017 – Vehicle Computation & Fleet Details. Establish private attributes, access modifiers, constructors, and encapsulated getter/setter methods with precondition checks.

**Reference Baseline / Scenario / Code:**

```
public class Q017_Vehicle_Computation_Details { private double amount; ... }
```

**STUDENT SOLUTION & CODE WORKSPACE:**

**[1] Class Attributes & Modifiers:** _____

**[2] Constructor Implementation:** _____

**[3] Encapsulation & Preconditions:** _____

## EXERCISE 17.2: CORE BUSINESS LOGIC & COMPUTATION (FUEL CONSUMPTION RATE FORMULA)

**Problem Statement:** Implement the core algorithmic method for Q017 – Vehicle Computation & Fleet Details. Formulate the calculation logic, boundary edge validations, and state updates.

**Reference Baseline / Scenario / Code:**

```
public double computeResult() { // Algorithm logic for Q017 – Vehicle Computation & Fleet Details return calculatedValue; }
```

**STUDENT SOLUTION & CODE WORKSPACE:**

**[1] Algorithm Steps & Formula:** _____

**[2] Conditional Branching Rules:** _____

**[3] Method Implementation Code:** _____

## EXERCISE 17.3: EDGE-CASE HANDLING & BOUNDARY SAFEGUARDS (PAYLOAD LOAD FACTOR CALCULATION)

**Problem Statement:** Identify 3 critical edge cases for Q017 – Vehicle Computation & Fleet Details (e.g. zero values, negative numbers, extreme thresholds). Implement defensive guard clauses and error handling.

**Reference Baseline / Scenario / Code:**

```
if (value <= 0) throw new IllegalArgumentException("Invalid input value");
```

**STUDENT SOLUTION & CODE WORKSPACE:**

**[1] Edge-Case Boundary Matrix:** _____

**[2] Defensive Guard Clauses:** _____

**[3] Validation Logic Implementation:** _____

## EXERCISE 17.4: TESTING & VERIFICATION EXECUTION (OPERATING COST PER KM OUTPUT)

**Problem Statement:** Construct a JUnit 5 test method verifying that Q017 – Vehicle Computation & Fleet Details computes expected results accurately across normal, boundary, and invalid scenarios.

**Reference Baseline / Scenario / Code:**

```
@Test void testCalculation() { assertEquals(expected, service.compute(input)); }
```

**STUDENT SOLUTION & CODE WORKSPACE:**

**[1] JUnit 5 Test Annotations:** _____

**[2] Assertion Statements:** _____

**[3] Verification Output Log:** _____

### SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **Encapsulation & Access:** All fields private; access mediated by validated methods.
- [ ] **Algorithmic Correctness:** Business logic computes exact expected outputs across all slabs.
- [ ] **Edge Safeguards:** Defensively guards against zero, negative, and extreme inputs.
- [ ] **Code Standards:** Adheres to standard Java naming conventions and clean code practices.
- [ ] **Test Verification:** JUnit test assertions verify deterministic execution.

### CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *How does encapsulation protect the internal state of Q017 – Vehicle Computation & Fleet Details from unauthorized external corruption?*
- *Why is it essential to test boundary values at the exact tariff or discount transitions?*

**Reflection Notes:**

_____

_____

| COMPLETION STATUS: | EVALUATOR & MENTOR REMARKS: |
|---|---|
| [ ] COMPLETED (Meets All Solution Criteria)<br>[ ] NEEDS REVISION (Resubmit Corrected Work)<br><br>**Evaluator Signature:** _____ | Feedback Notes:<br>_____<br>Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Edge Cases |